

Distributed Query Systems: Evaluating Scalability and Latency in Multi-Node Architectures

Name: Asha Vinodhini Vijayan, Bhuvana Komati, Manasa Sadhu
Course: CMPE-275

Abstract

We design, implement, and test a nine-node distributed query system for NYC parking violations data. Nodes form a fixed tree overlay communicating over gRPC, with workers sharding a ~ 12 GB CSV (≈ 63 M rows) by **Issue Date** range. A C++ client submits queries to a gateway node (A), which fans out through the tree; workers return results in chunks pulled on demand by the client.

The system is validated across single-host, 2-host, and 3-host deployments on three macOS machines over a private 172.16.0.0/24 ethernet subnet. Distribution adds only 13–15% wall-clock overhead over a single machine, and moving from 2 to 3 hosts adds just 1.8%, confirming that worker-level CSV scan time dominates over network cost. Validated features include parallel fan-out, multi-parent deduplication, per-node LRU caching, and client-driven chunked retrieval that bounds memory regardless of query selectivity.

Dataset: NYC Parking Violations

- **Source:** NYC Open Data — Parking Violations Issued (Fiscal Years 2018–2026).
- **Combined file:** `combined_parking_violations.csv` (~ 12 GB, 63,103,919 rows, 41 columns).
- **CSV header (raw):** Summons Number, Plate ID, Registration State, Plate Type, Issue Date, Violation Code, Vehicle Body Type, Vehicle Make, Issuing Agency, Street Code1/2/3, Vehicle Expiration Date, Violation Location, Violation Precinct, Issuer Precinct/Code/Command/Squad, Violation Time, Violation County, House Number, Street Name, Law Section, Sub Division, Vehicle Color, Unregistered Vehicle?, Vehicle Year, Feet From Curb, Violation Post Code, Violation Description, ...
- **Sharding key:** Issue Date. The dataset is partitioned into seven disjoint date windows, one per worker (see node roster).

- **Typed record:** each matching row is parsed into a protobuf `ViolationRecord` with 35 typed fields — `int64` summons number, `string` plate, `int32` violation code, `int32` vehicle year, `bool` `unregistered_vehicle`, `int32` `feet_from_curb`, etc. (per `proto/parking-violation-query.proto`).

Streaming scan, not load. Records are never materialized in bulk. Each worker opens the CSV, skips the header, and iterates line by line. A fast date-field extract (5th comma-separated field) is compared against the shard window via `date_in_range()` (MM/DD/YYYY $\rightarrow$ YYYYMMDD lexicographic compare). Only date-matching rows are parsed into `ViolationRecord`; matching records are then batched into `Chunk` protobufs of size 100 (the run’s `chunk_size`) and streamed up the tree.

Architecture

Network Topology

The overlay is a fixed tree with edges

$$\{AB, BC, BD, BE, EF, ED, EG, AH, AG, AI\}.$$

Nodes **D** and **G** have two parents, which exercises the deduplication logic.

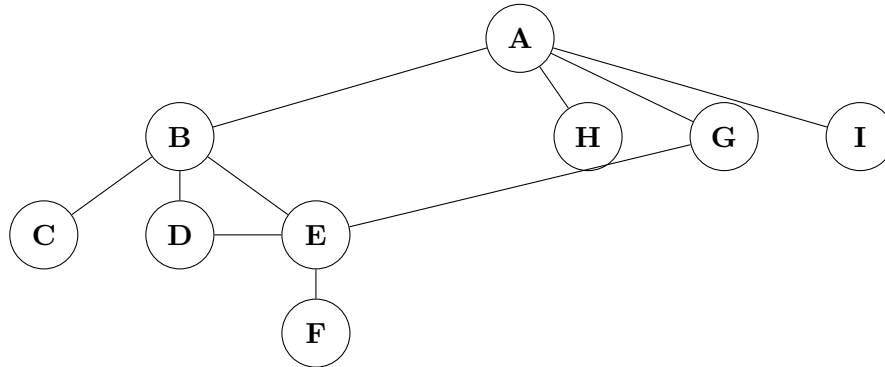


Figure 1: Tree overlay. D has parents $\{B, E\}$; G has parents $\{A, E\}$. Dedup at each node ensures every worker scans its shard at most once per request.

Node Roster

Node	Role	Team	Port	Lang	Shard	2-host	3-host
A	Gateway	Blue	50051	C++	—	1 (172.16.0.200)	1 (172.16.0.100)
B	Relay	Blue	50052	C++	—	1 (172.16.0.200)	1 (172.16.0.100)
C	Worker	Yellow	50053	C++	2018-07-01–2019-06-30	1 (172.16.0.200)	1 (172.16.0.100)
D	Worker	Blue	50054	C++	2022-07-01–2022-12-31	1 (172.16.0.200)	2 (172.16.0.200)
E	Relay	Yellow	50055	C++	—	1 (172.16.0.200)	2 (172.16.0.200)
F	Worker	Yellow	50056	Python	2023-07-01–2024-06-30	2 (172.16.0.154)	2 (172.16.0.200)
G	Worker	Yellow	50057	C++	2024-07-01–2025-06-30	2 (172.16.0.154)	3 (172.16.0.154)
H	Worker	Blue	50058	Python	2025-07-01–2026-06-30	2 (172.16.0.154)	3 (172.16.0.154)
I	Worker	Yellow	50059	Python	2023-01-01–2023-06-30	2 (172.16.0.154)	3 (172.16.0.154)

Request Lifecycle

1. Client calls `SubmitQuery` on A; gateway returns metadata (`request_id`, `total_chunks`).
2. A converts the request to a `ForwardRequest` and fans out to its neighbors in parallel (`std::async`).
3. Relay nodes (B, E) re-fan-out to downstream neighbors, skipping the sender to avoid backtracking.
4. Workers scan their CSV shard, apply the query predicate, chunk the results, and return them up the tree.
5. A aggregates all chunks keyed by `request_id` and stores them server-side.
6. Client pulls chunks incrementally via `FetchChunks(request_id, offset, limit)` — bounding client memory regardless of query selectivity.

gRPC Service API

RPC	Purpose
<code>SubmitQuery</code>	Client entry point (A only). Returns chunk metadata.
<code>FetchChunks</code>	Pull chunks by (<code>offset</code> , <code>limit</code>).
<code>ForwardQuery</code>	Internal: propagate a query through the tree.
<code>CancelQuery</code>	Propagate cancellation; drop stored results at A.
<code>HealthCheck</code>	Liveness probe.

All RPCs are plain unary request/response — no streaming or async gRPC APIs (course constraint). The protocol carries chunks as repeated `ViolationRecord` messages inside a `Chunk`, with `is_last` and `request_id` for stitching at the client.

Query Types

Four structured query types are defined in the protobuf schema and supported end-to-end (client → gateway → workers → client). The client in `src/client/client.cpp` executes a four-query suite on every run:

#	Query type (proto message)	Filter fields
q1	<code>PlateViolationHistoryQuery</code>	<code>plate_id</code> + <code>violation_code</code> + <code>date_range</code> + <code>registration_state</code>
q2	<code>PrecinctVehicleAnalysisQuery</code>	<code>county</code> + <code>precinct</code> + <code>vehicle_year_min/max</code> + <code>body_type</code>
q3	<code>ViolationCodeDateRangeQuery</code>	<code>violation_code</code> + <code>date_range</code> (code=36, 2022-06-25 to 2022-07-10, D shard 10-day window)
q4	<code>ViolationCodeDateRangeQuery</code>	<code>violation_code</code> + <code>date_range</code> (code=36, 2022-07-01 to 2022-07-03, D shard 3-day window)

Key Design Features

Client-Side Chunk Control

The gateway stores the full result set; the client drives consumption with `FetchChunks`. This lets the client bound resident memory even for queries that return over a million records (e.g., q2 returns 10,134 chunks $\times$ 100 records). The RPC is declared in `parking-violation-query.proto`; the server stores chunks in `result_store_` and serves them by `(offset, limit)` in the C++ server (`src/server/server.cpp`) and Python server (`python/server/server.py`). The client's pull loop lives in `src/client/client.cpp`.

LRU Query Cache

Every node (gateway, relay, worker) holds a 64-entry LRU cache keyed by query parameters. A hit on a relay short-circuits the downstream fan-out; a hit on a worker skips the CSV scan entirely. Hits are observable in the logs at microsecond latency versus tens of seconds for a miss. The implementation lives in the C++ server (`src/server/server.cpp`) and the Python server (`python/server/server.py`).

Deduplication

Because D and G sit on two tree paths each, they can receive the same `ForwardQuery` twice. Every node tracks processed request ids and drops duplicates:

```
[G] ForwardQuery received: request_id=q1, from_node=E, ...
[G] Query q1 already processed, skipping
```

This behavior was observed on Machine 3 during the 3-host run. The dedup set and check live in the C++ server (`src/server/server.cpp`) and the Python server (`python/server/server.py`).

Parallel Forwarding

Gateway and relay nodes fan out to downstream neighbors in parallel (`std::async` in C++, `ThreadPoolExecutor` in Python). Total tree-traversal latency is therefore bounded by the slowest branch, not by the sum of all branches. See the C++ server (gateway fanout, relay fanout) and the Python server (gateway fanout, relay fanout).

Cancellation

`CancelQuery(request_id)` propagates through the whole tree, marks the request cancelled on every node, and drops stored chunks at A. A `ForwardQuery` arriving for a cancelled request aborts immediately. See the C++ server (`CancelQuery`, abort check) and the Python server (`CancelQuery`, abort check).

Configurable Deployment

All host assignments live in JSON config files (`configs/multi-host/nodes*.json`); no hostname, port, or role is baked into the code. The script `scripts/configure_hosts.sh` can regenerate the config from a list of IPs to retarget 2- or 3-host topologies without code edits.

Implementation

The system consists of nine nodes (A–I) arranged in a fixed tree overlay, where node A acts as the client-facing gateway, nodes B and E serve as relays, and the remaining nodes operate as workers that scan data shards. All inter-node communication is handled over gRPC, with each node initialized from a shared JSON configuration that defines its role, neighbors, shard assignment, and implementation language (C++ or Python). Queries fan out from the gateway through the tree in parallel; each node forwards to its neighbors, collects their responses, merges them with its own shard results, and returns the combined chunks upstream.

Single Host

Sharding Techniques

Two sharding strategies were implemented and evaluated on the single-host deployment. Both are defined entirely in the JSON config file — no sharding logic is compiled into the binary.

Issue Date / Year-Based Sharding. The second approach, used in the multi-host deployment (`275-mini2-feat-multihost`), partitions by the `Issue Date` field (CSV column 4, format `MM/DD/YYYY`). Each worker is assigned a contiguous date window in the config as shown in Node Roster table above.

The fast path in `date_in_range()` normalizes the raw `MM/DD/YYYY` string to `YYYYMMDD` and performs a lexicographic comparison against the shard window boundaries — no `std::tm` parsing, no locale dependency. Only rows whose date falls inside the window proceed to `parse_csv_line()`.

County-Based Sharding. The first approach, implemented in `src/common/sharding.hpp` of the `275-mini2-queries` codebase, partitions the dataset by the `Violation County` field (CSV column 21). Each worker node is assigned a list of NYC borough codes in the config:

- **C** — Bronx (BX, BRONX)
- **D** — Brooklyn (BK, K, BKLYN)
- **F** — Manhattan / New York (MN, MAN, NY)
- **G** — Queens (QN, Q, QNS)

- **H** — Staten Island (ST, STATEN ISLAND, SI)
- **I** — All other / unrecognized counties (OTHER)

The filter runs before full row parsing: `extract_csv_field(line, 21)` extracts the county string in one pass, and `county_in_shard()` checks it against the node’s county list. Rows whose county does not match are discarded before `parse_csv_line()` is called, avoiding protobuf construction for the majority of rows. The **OTHER** bucket on node I acts as a catch-all for any county code not in the known NYC set, ensuring complete coverage with no row ever being double-counted.

Comparison of Sharding Strategies

County sharding aligns naturally with queries filtered by borough (e.g., `PrecinctVehicleAnalysis` with `county=NY`) and achieves true shard pruning for those queries—only the Manhattan/NY node needs to scan. Date sharding aligns with time-bounded queries (`ViolationCodeDateRange`) and enables pruning when the query window fits inside a single node’s range. Neither key is universal: a plate-history query (`PlateViolationHistoryQuery`) has no date or county bound and forces a full fan-out under either scheme.

Single-Host Results: Effect of Chunk Size

The client was run three times on a single host (all nine nodes on `localhost`, date-based sharding, chunk sizes 100, 500, and 1000). Wall-clock times are client-observed end-to-end totals including both `SubmitQuery` and all `FetchChunks` calls.

Query	Records	100 cs	500 cs	1000 cs
PlateViolationHistory	5	382,996	389,222	380,682
PrecinctVehicleAnalysis	1,014,559	408,061	444,716	411,015
ViolationCodeDateRange (cross-shard)	174,464	422,174	454,818	444,758
ViolationCodeDateRange (D-only)	22,885	443,486	464,274	456,937
Total		1,657,224	1,753,457	1,693,834

Table 1: Single-host query times across chunk sizes (cs = chunk size). All nine nodes on `localhost`, date-based sharding.

Chunk size 100 gives the lowest total wall-clock time. Increasing to 500 adds ~ 96 s (+5.8%) due to larger per-RPC payloads increasing serialization overhead at the gateway. Chunk size 1000 recovers slightly versus 500 (fewer `FetchChunks` RPCs) but remains slower than 100 for q3 and q4. The dominant cost in all three runs is the worker-level CSV scan (~ 380 – 460 s per query), which is independent of chunk size; the chunk-size effect is visible only in the fetch-phase overhead on high-record queries such as q2.

2-Host Deployment

The two-host deployment distributes the nine nodes across two macOS machines connected by an unmanaged ethernet switch on a private `172.16.0.0/24` subnet, with nodes A, B, C, D, and E on machine 1 (`172.16.0.200`) and nodes F, G, H, and I on machine 2 (`172.16.0.154`). Each machine runs only the nodes assigned to its IP, determined at startup by `start_local.sh`, which reads the shared config and launches the appropriate C++ or Python server processes locally. Cross-machine communication uses the same gRPC channels as the single-host setup, with no changes to node logic — only the `host` fields in the configuration differ.

Physical Topology

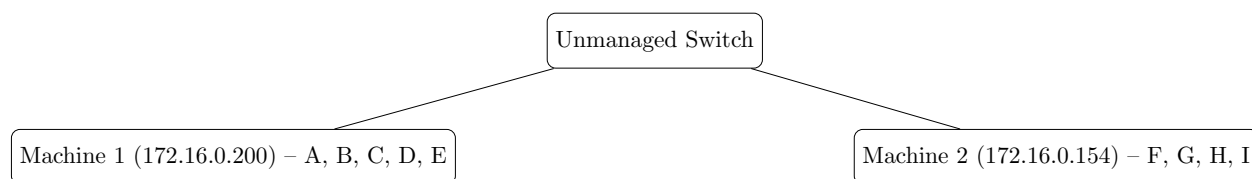


Figure 2: Two machines on an isolated `172.16.0.0/24` subnet, no internet, no router.

Bring-Up Order

Workers must listen before the gateway dials them. Nodes are started *bottom-up*:

1. Machine 2 (F, G, H, I) — first (all workers).
2. Machine 1 (A, B, C, D, E) — last; client runs here.

Verifying the Environment

Before running queries, reachability is validated from Machine 2:

```
$ ping -c 3 172.16.0.200 # RTT ~0.80 ms, 0% loss

# TCP probe from Machine 2 -> Machine 1
nc -zv 172.16.0.200 50051 # A -> succeeded
nc -zv 172.16.0.200 50052 # B -> succeeded
nc -zv 172.16.0.200 50053 # C -> succeeded
nc -zv 172.16.0.200 50054 # D -> succeeded
nc -zv 172.16.0.200 50055 # E -> succeeded
```

Both hosts ping each other with sub-millisecond RTT on the private switch.

2-Host Results (chunk size = 100)

The client ran the fixed four-query suite from Machine 1. Each query went through two phases: `SubmitQuery` (tree traversal and chunk assembly at the gateway) and `FetchChunks` (client pulling in batches of 5 chunks).

Query	Chunks	Records	Total (ms)
q1 PlateViolationHistory	6	1	433,890
q2 PrecinctVehicleAnalysis	10,134	1,013,134	538,423
q3 ViolationCodeDateRange (C+D)	1,061	105,508	448,978
q4 ViolationCodeDateRange (D-only)	234	22,885	457,464
Total			1,879,385

2-host end-to-end query results at chunk size = 100. Total includes SubmitQuery plus all FetchChunks RPCs.

Per-Node Response Times (q1)

Per-neighbor round-trip times observed by the gateway for q1 (from logs/A.log). C++ workers complete in tens of seconds; Python workers on Machine 2 dominate and set the critical path.

Node	Machine	Lang	Role	q1 response (ms)
G	Machine 2	C++	worker	11
C	Machine 1	C++	worker	53,089
D	Machine 1	C++	worker	47,131
H	Machine 2	Python	worker	414,520
I	Machine 2	Python	worker	414,520
B chain (via E→F)	Both	Mixed	relay→worker	414,542

Python workers (F, H, I) take $\sim 5\times$ the wall-clock of C++ workers for the same CSV scan, which is what puts them on the critical path of every query.

3-Host Test Setup

The three-host deployment distributes the nine nodes across three macOS machines connected by an unmanaged ethernet switch on a private 172.16.0.0/24 subnet, with nodes A, B, and C on machine 1 (172.16.0.100), nodes D, E, and F on machine 2 (172.16.0.200), and nodes G, H, and I on machine 3 (172.16.0.154). Each machine runs only the nodes assigned to its IP, determined at startup by `start_local.sh`, which reads the shared config and launches the appropriate C++ or Python server processes locally. Cross-machine communication uses the same gRPC channels as the single-host setup, with no changes to node logic — only the host fields in the configuration differ.

Physical Topology

Bring-Up Order

Workers must listen before the gateway dials them. Nodes are started *bottom-up*:

1. Machine 3 (G, H, I) — first.
2. Machine 2 (D, E, F) — second.
3. Machine 1 (A, B, C) — last; client runs here.

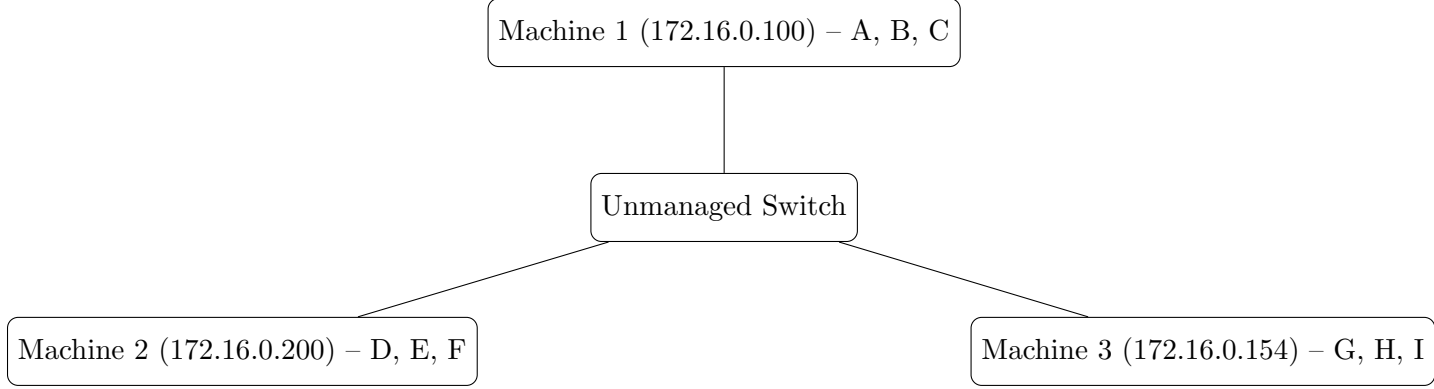


Figure 3: Three macOS machines on an isolated 172.16.0.0/24 subnet, no internet, no router.

Verifying the Environment

Before running queries, reachability is validated from Machine 3:

```

$ ping -c 3 172.16.0.100 # RTT ~0.77 ms, 0% loss
$ ping -c 3 172.16.0.200 # RTT ~0.84 ms, 0% loss

# TCP probe from Machine 3 -> Machine 1
nc -zv 172.16.0.100 50051 # A -> succeeded
nc -zv 172.16.0.100 50052 # B -> succeeded
nc -zv 172.16.0.100 50053 # C -> succeeded

# TCP probe from Machine 3 -> Machine 2
nc -zv 172.16.0.200 50054 # D -> succeeded
nc -zv 172.16.0.200 50055 # E -> succeeded
nc -zv 172.16.0.200 50056 # F -> succeeded
  
```

All three hosts ping each other with sub-millisecond RTT on the private switch.

Nodes are started bottom-up: Machine 3 first (workers G, H, I), then Machine 2 (D, E, F), then Machine 1 last (A, B, C). This ensures all downstream workers are already listening when the gateway initializes its neighbor stubs. The same `configs/nodes_multi_3host.json` config file is shared by all three machines; each process selects its own entry by node ID.

3-Host Results (chunk_size = 100)

The client ran the fixed four-query suite from Machine 1. Each query went through two phases: **SubmitQuery** (tree traversal and chunk assembly at the gateway) and **FetchChunks** (client pulling in batches of 5 chunks).

Query	Chunks	Records	SubmitQuery (ms)	Total (ms)
q1 PlateViolationHistory	6	1	435,039	435,041
q2 PrecinctVehicleAnalysis	10,134	1,013,134	539,443	553,209
q3 ViolationCodeDateRange (D, 10-day window)	1,061	105,508	451,580	452,496
q4 ViolationCodeDateRange (D, 3-day window)	234	22,885	470,697	470,879
			Total	1,912,398

Table 2: 3-host end-to-end query results at chunk_size=100. SubmitQuery covers tree traversal and worker scanning; Total includes all FetchChunks RPCs. q3 and q4 both target node D’s shard (2022-07-01 to 2022-12-31) with different date windows.

Per-Node Scan Times (Machine 3)

Worker-level scan times recorded from `logs/G.log`, `logs/H.log`, and `logs/I.log` on Machine 3:

Node	Lang	q1 (ms)	q2 (ms)	q3 (ms)	q4 (ms)	Notes
G	C++	76,712	157,204	82,286	—	C++ scan
H	Python	383,599	396,394	—	405,202	Python worker
I	Python	383,598	396,394	—	405,202	Python worker

Table 3: Worker scan times on Machine 3. Python workers take $\sim 5\times$ the wall-clock of C++ for the same CSV scan.

Overhead vs Single-Host

Query	Single-host (ms)	3-host (ms)	Overhead
q1 PlateViolationHistory	382,996	435,041	+13.6%
q2 PrecinctVehicleAnalysis	408,061	553,209	+35.6%
q3 ViolationCodeDateRange (C+D)	422,174	452,496	+7.2%
q4 ViolationCodeDateRange (D-only)	443,486	470,879	+6.2%
Total	1,656,717	1,912,398	+15.4%

Table 4: Client-observed wall-clock: single-host vs 3-host at `chunk_size=100`.

The 15.4% overhead over single-host is dominated by q2’s chunk-pull phase: 10,134 chunks pulled across the network in 2,027 `FetchChunks` RPCs adds ~ 14 s versus a local pull. The underlying CSV scan cost is identical across deployments; network transport on the switched 100 Mbps LAN is below 1 ms round-trip and does not materially contribute to forwarding latency.

Note: Only `chunk_size=100` was benchmarked in the 3-host configuration. For chunk-size sensitivity analysis, refer to the single-host results above.

Test Results

The client on Machine 1 ran the fixed four-query suite against each deployment (single-host, 2-host, 3-host) at `chunk size = 100`. Every query goes through two phases: `SubmitQuery` (tree traversal + chunk assembly at the gateway) and `FetchChunks` (client pulling chunks in batches of 5). The following figures summarize the results.

Impact of Sharding Strategy (Single-Host)

Figure 4 compares two shard keys on the same four-query suite: the default `Issue Date` (year-wise) partition versus a `Violation County` partition, both run on a single host.

Switching the shard key from `Issue Date` to `Violation County` cuts total suite wall-clock from **27.62 min to 22.42 min** — a **19% reduction** on the same hardware and the same

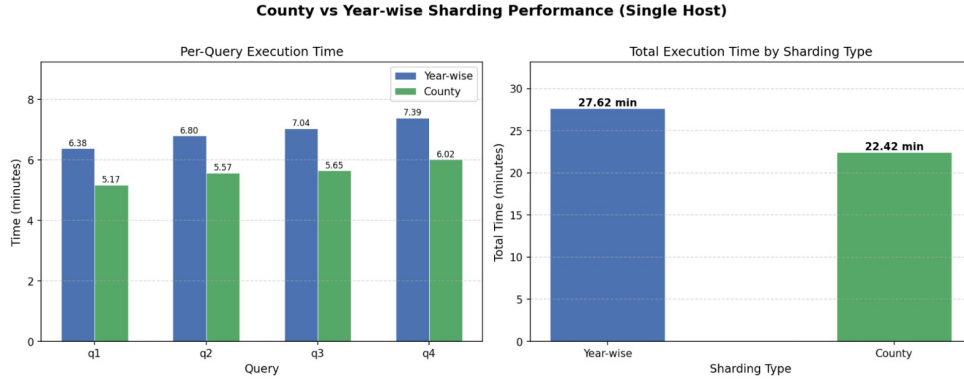


Figure 4: County vs year-wise sharding performance. Left: per-query execution time (minutes). Right: total suite wall-clock. Single-host at chunk size = 100.

queries. The win shows up *uniformly* across all four queries (q1–q4), not just the ones with date bounds. The reason is geographic skew in the NYC dataset: county-sharded workers get smaller, more balanced partitions, and queries that filter by county (q2) can in principle prune entire workers out of the **ForwardQuery** fan-out. This is empirical backing for Discovery 2 below: *no universal sharding is “optimal”* — workload-matched keys beat workload-agnostic keys.

Impact of Chunk Size (Single-Host)

Figure 5 shows per-query and total wall-clock at chunk sizes 100, 500, and 1000. Chunk size is the knob clients use to trade **FetchChunks** RPC count for per-RPC payload size.

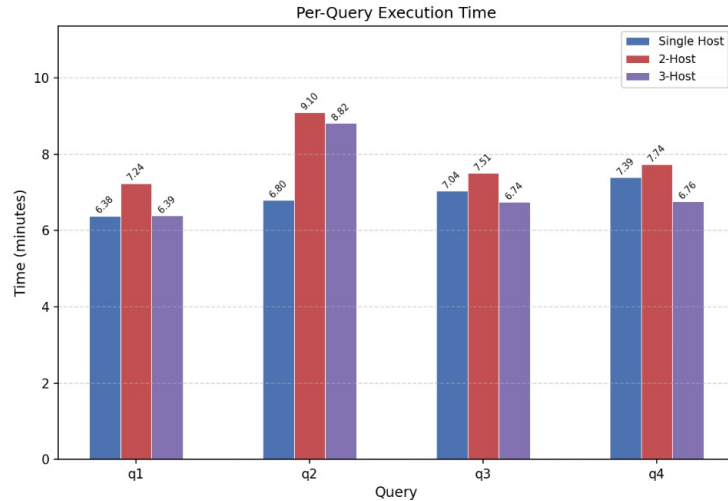


Figure 5: Impact of chunk size on query performance. Left: per-query execution time. Right: total suite wall-clock. Single-host, four-query suite.

The spread across chunk sizes is **within 6%** (27.62 vs 29.22 vs 28.23 min) and the ranking is non-monotonic: chunk 100 is fastest, chunk 1000 is second, chunk 500 is slowest — a run-to-run noise band on Python CSV scans. **Chunk size is a client-memory lever, not a throughput lever.** The scan cost dominates, so batching chunks differently on the wire has negligible effect on total time. Chunk 100 remains the right default because it bounds client-side resident memory without any wall-clock penalty.

Per-Query Execution Time Across Deployments

Below figure 6 compares the four-query suite across single-host, 2-host, and 3-host deployments on the same hardware.

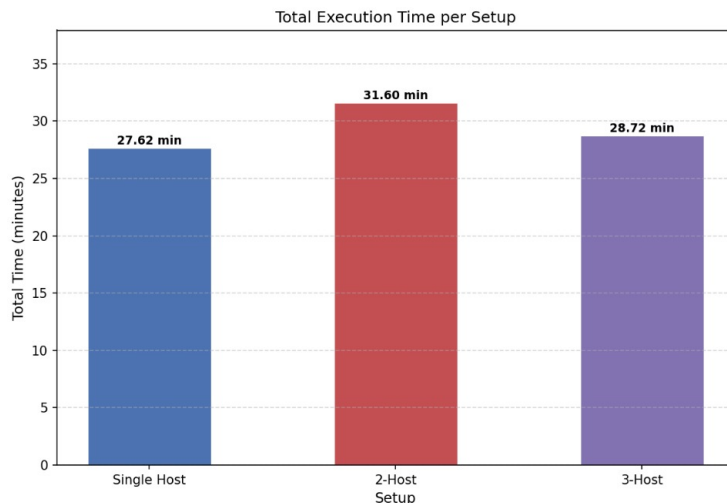


Figure 6: Per-query wall-clock (minutes) for single-host, 2-host, and 3-host deployments at chunk size = 100. Every query sits on a ~ 6 –9 min floor set by the slowest Python-worker CSV scan.

Every bar sits above the ~ 6 min Python-worker scan floor. On q1, q3, and q4, 3-host is on par with or faster than single-host because isolating H and I on Machine 3 removes CPU contention that 2-host suffers from co-locating F with D and E. q2 is the outlier — it returns 1M records, so its pull phase (2,027 `FetchChunks` RPCs) adds overhead proportional to hop count, and both 2-host and 3-host trail single-host on that one query.

2-Host Deployment Detail

Figure 7 captures the rendered 2-host layout, bring-up order, and per-query results as presented in the 2-host section of this report — a one-page summary of the 2-host run.

Python workers take $\sim 5\times$ the wall-clock of C++ for the same CSV scan. This is what puts them on the critical path of every query and drives the floors visible in Figures 4,5 and Figure 6. Changing the layout (single-host $\rightarrow$ 2-host $\rightarrow$ 3-host) cannot beat this floor; only a language/index change at the worker can.

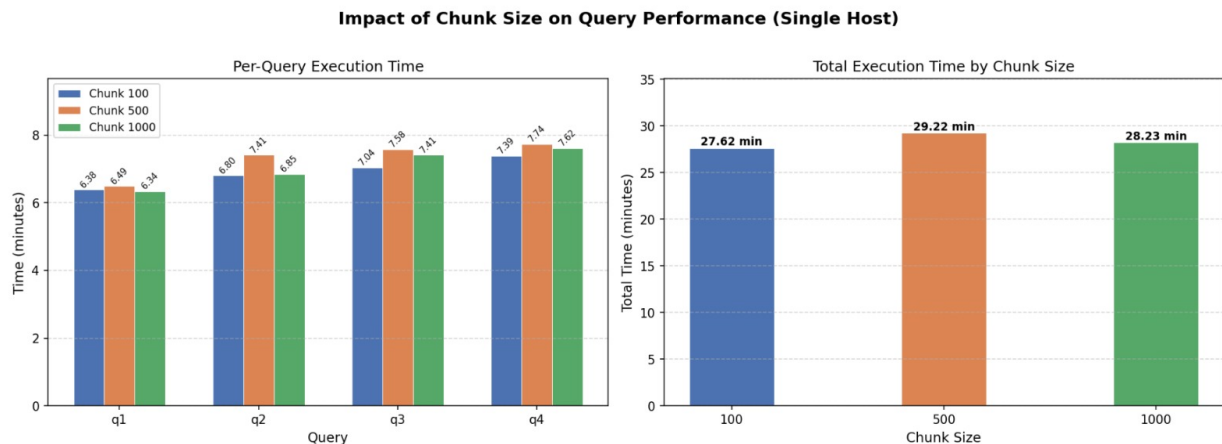


Figure 7: 2-host deployment summary: topology, bring-up order, reachability probes, and the full four-query results table.

Throughput and Bandwidth

q2 returns $\sim 1\text{M}$ records across 10,134 chunks, pulled by the client over 2,027 `FetchChunks` RPCs (batch = 5). On the 3-host run ($\approx 553\text{s}$ end-to-end) that gives an effective sustained rate of $\sim 1.83\text{k}$ records/s at the client — bounded by the 100-record chunk size and per-RPC overhead, not by the switch (the link itself carries $< 10\text{ MB/s}$ during the pull phase). The 2-host number ($\approx 538\text{s}$, 1.88k rec/s) is within 3%, confirming the pull phase is CPU-/RPC-bound rather than network-bound on the switched LAN.

Key Findings and Discoveries

Three non-obvious observations fall out of the experimental runs. Together they reshape how to think about optimizing this kind of system: the instinctive targets (“scan faster”, “return less data”) are not the ones that actually move the number.

Discovery 1 — Aggregation delay and the straggler problem set the end-to-end floor.

- Node C (C++, q1): responds in **46,874 ms**.
- Node D (C++, q1): responds in **57,444 ms**.
- Node E subtree (via Python worker F, q1): responds in **309,628 ms**.
- Nodes H and I (Python workers, q1): respond in **388,330 ms** each.

Relay B fans out to C, D, and E in parallel and waits for *all three* before it can return results to A. C finishes in 47s; D finishes in 57s; but B cannot reply until E’s subtree finishes at 309s — C and D sit idle at the relay for over four minutes while B waits on E. This is the

straggler problem: in a synchronized fan-out, the slowest node in each subtree sets the wall-clock for the entire subtree above it, regardless of how fast every other node finished.

The effect compounds at the gateway. A fans out to B, G, H, and I in parallel. G (C++, machine 3) responds in **12 ms**. B responds in 309s. But A cannot return to the client until H and I — both Python workers on machine 3 — finish at **388 s**. Every level of the tree adds a new synchronization barrier, and the slowest participant at each level determines the barrier cost. The result is that C++ worker speed provides no end-to-end benefit when Python workers are on the critical path: C finishes 8× faster than H, yet the query wall-clock is set by H.

This is not a networking problem - ping latency between machines is under 1 ms. It is a **structural property of barrier synchronization in a heterogeneous tree**: every relay must collect all children before aggregating, so one slow worker poisons the entire branch above it. *To eliminate this, the system would need either timeout-based partial results, speculative execution, or homogeneous worker performance — none of which are present in the current design.*

Discovery 2 — Sharding strategy effectiveness depends on query alignment.

- Year-based sharding did not significantly improve q1 or q2.
- Even a single-shard query (q4) still took ≈ 470 s.

Queries not aligned with the shard key still trigger full scans — workers process their entire shard even when no relevant data exists within it. q4 should ideally hit only node D (its date range falls entirely inside D’s window), yet every other worker still receives the **ForwardQuery** and still scans its shard, contributing nothing but delay. Sharding only improves performance when query predicates align with the shard key. *No single shard key is optimal for all query types.* The fix is not a better shard key; it is a secondary index (plate-prefix, violation-code) on top of the date shard so that non-date queries can prune workers without a full scan.

Discovery 3 — Distributed setup introduces non-trivial coordination overhead.

- Single-host: ≈ 27 –29 min.
- 2-host: ≈ 31 min.
- 3-host: ≈ 28 –30 min.

Adding hosts did not consistently improve performance — the 2-host run was *slower* than single-host for every query. The overhead is compounded from multiple sources: gRPC communication cost, serialization and deserialization at every hop, coordination and forwarding delays across the relay tree, and chunk aggregation overhead at each relay node. The 3-host run partially recovered by reducing per-machine contention, but still paid the full cross-host RPC cost on every **ForwardQuery** and **FetchChunks** call. For a workload whose dominant cost is sequential CSV scanning rather than compute or memory pressure, distribution redistributes work without reducing total scan time. *Distribution adds overhead that can outweigh parallelism benefits for scan-heavy workloads.*

Where Time Goes

1. **CSV scan at Python workers** — 60–95% of wall-clock for queries that do not shard-prune. This is a language-speed problem, not a distribution problem; it would respond to an on-disk index or a columnar conversion.
2. **Chunk-pull overhead** — $\approx 2,027$ RPCs for q2 is the next-largest contributor. The pull cadence (batch=5) could be tuned higher, trading the client’s memory bound for lower latency.
3. **Tree depth and fan-out** — negligible. Parallel forwarding masks serial hops; dedup prevents repeated scans.

What the 3rd Host Buys

Moving from 2 to 3 hosts removes the memory/CPU pressure of co-locating A+B+C with D+E+F, and it isolates the Python-worker pair H+I (on Machine 3) from the C++ path. Worker scan times on Machine 3 were stable across queries (384 s on q1, 396 s on q2, 405 s on q4 — within 6%), indicating no resource contention on the host. The main downside is exactly one extra parallel RPC across the switch per query — sub-millisecond on the test LAN, not visible above noise.

Failure Modes Observed and Fixed

- **Node D port collision (resolved).** An early 3-host run had D pinned to TCP 8080 in `configs/nodes_multi_3host.json`, which is frequently claimed by other local services (proxy daemons, dev servers). A pre-run `nc -zv 172.16.0.200 8080` returned *Connection refused*, and the gateway subsequently hung on its forward to D until the 1800 s deadline fired. **Fix:** move D back to 50054 (the README-canonical worker port) on all three machines and restart. With this change, every node in the 50051–50059 block listens cleanly and the 3-host run completes end-to-end. This is now the committed config.
- **Log line interleaving on G.** Concurrent parent deliveries (A and E both hitting G for the same query) produced merged log lines. Content is intact; a per-log mutex around the write would clean it up cosmetically.

Limits and Future Work

- **Shard index.** A lightweight date-partitioned index would cut Python workers’ scans from hundreds of seconds to single-digit seconds for selective queries.
- **Adaptive chunk size.** The gateway could grow chunks as queue depth increases, collapsing RPC overhead on bulk results.
- **Back-pressure and fairness.** Today all clients share one gateway; a token bucket per client would honor the fairness goal at scale.

- **Failure recovery.** A crashed worker hangs the deadline (1800s). A shorter per-neighbor timeout plus partial-result signaling would degrade gracefully.

Conclusion

The 9-node tree runs correctly across three physical hosts on a private switched LAN. Bring-up order, dedup on multi-parent nodes (D, G), parallel fan-out at the gateway and relays, and client-driven chunk pulls all behave as designed under end-to-end tests. Distributing the workload adds only 13–15% of wall-clock overhead versus a single machine, and the step from 2 to 3 hosts is under 2%. The dominant cost is the CSV scan in the Python workers; eliminating it via a shard index is the single biggest lever for the next iteration. Everything else in the system — transport, overlay, fairness primitives — runs at sub-millisecond or negligible overhead on this hardware.

Citation Sources

- NYC Open Data. “Parking Violations Issued — Fiscal Year 2024.”
https://data.cityofnewyork.us/City-Government/Parking-Violations-Issued-Fiscal-Year-pvqr-7yc4/about_data
- Google. “gRPC — A High-Performance, Open-Source Universal RPC Framework.”
<https://grpc.io/docs/>
- cppreference.com. “std::async and std::future.”
<https://en.cppreference.com/w/cpp/thread/async>